

Distributed Databases

→ Databases must be autonomous: Each DB should be able to function without another

→ Semi-join strategy:

→ Compute $\Pi_J(R)$

→ " $S \bowtie \Pi_J(R)$

→ " $R \bowtie (S \bowtie \Pi_J(R))$

$$\begin{aligned} R \bowtie (S \bowtie \Pi_J(R)) \\ &= (R \bowtie \Pi_J(R)) \bowtie S \\ &= R \bowtie S \\ &\text{(Pf. of correctness)} \end{aligned}$$

→ Cost estimates:

→ Assume communication cost = $C_1 + C_2 d$

→ " disk I/O " = $C_3 d$

↖ No. of bytes

$$d = N_R S_R, \text{ where } N_R = \text{no. of tuples in rel. } R \\ S_R = \text{size " " " " "}$$

→ Guaranteeing ACID: Local ACID $\nrightarrow$ Global ACID

→ 2PC (2-Phase Commit) protocol: guarantees global atomicity

→ Master sends START WORK to cohorts along with the associated work.

→ A cohort will send a WORK DONE after completion

→ Phase 1: Obtaining a decision:

→ Master asks all cohorts to prepare to commit T.

→ Transaction manager at each cohort will determine if T can be committed in the first place.

→ If cannot be committed, add $\langle \text{abort } T \rangle$ to log and send abort message to master.

→ If can be committed:

→ add $\langle \text{ready } T \rangle$ to the log

→ flush all log records of T

→ send ready message to master.

→ Phase 2: Recording the decision

- T can be committed only if all cohorts send **ready**.
- Master adds decision log record, i.e. **<ready T>** or **<abort T>** to the log & flushes it.
- Master sends commit or abort msg. to cohorts, who write their log records & reply with an **ack**.
- After all cohorts' log is recd, master adds **<end T>** to the log and then "forgets" the transaction.

→ Failures:

→ Cohort failure: Upon recovery, cohort examines its log:

- If log has **<commit T>**: **redo(T)**
- " " **<abort T>**: **undo(T)**
- " " **<ready T>**: ask master for verdict or abort/commit
- " " no control records w.r.t. T: **undo(T)**

→ Master failure: cohort does not hear from master within timeout

→ If log does not have **<ready T>**: **abort(T)**

→ If log has **<ready T>**: wait for master

→ We may also ask sibling cohorts, if they know a **commit** or **abort**, do the same thing.

→ Network partition failure

→ Sites who have lost connection would initiate the master failure protocol.

→ Master, and sites which are still connected to master, will think other cohorts have failed, and follow the usual commit protocol.

→ Blocking problem: When all cohorts are ready to commit, but need to wait for the master to make the final decision.

→ Because locks taken by the transaction are still held, other transactions may be blocked.

- 3 Phase Commit - Avoids blocking
 - Assumptions:
 - No network partitioning (failure)
 - At most k sites can fail
 - Choice of k :
 - Large value: less chance of blocking
 - Small " : " delay in normal processing
- Concurrency control
 - Basic 2PL is not enough, strict 2PL is reqd., assuming no data replication.
 - If replication is there, we can use numerous techniques for concurrency control (read only write all / primary copy / majority vote)
- Centralised handling
 - Maintain a global wait-for graph at master.
 - Requires cohort lock managers to communicate with the master, even if the transaction is disjoint.
- Fully distributed handling: Each site has a local wait-for graph, which can be combined.
- Phantom deadlocks: A deadlock 'victim' is chosen even if the deadlock has been solved.
 - Due to delay b/t deadlock detection & resolution

Data Mining - APRIORI

- Interesting patterns:
 - Association → Clustering → Classification → Derivation
- find all rules $X \rightarrow Y$ (c, s), s.t. $X \cap Y = \emptyset$. $c = P(X|Y)$, $s = P(X \cup Y)$, where $c > \text{min-confidence}$, $s > \text{min-support}$

→ Simple soln: let a counter for all subsets. Find those sets are increment the counter for each of them.

→ Most items (in a market, for e.g.) will not satisfy the min. support condn., which is why we endup doing useless calculations.

→ Apriori Algorithm

→ Procedure:

→ Multiple scans of database

→ In scan i

→ Read database row-by-row & count occurrence of candidate item sets with length i

→ After the scan, determine the frequent itemsets of length i , i.e. F_i

→ Determine the candidate itemsets for $(i+1)^{th}$ iteration

→ Return $F_1 \cup F_2 \dots \cup F_m$

→ Poss 1: Count occurrence of all individual items

Determine F_1

" candidate itemsets for next scan by $(F_1 \times F_1)$

→ Poss 2: Count occurrences of 2-item sets by storing counter on a 2-D strictly lower triangular matrix

Diagonal elements are 0

Determine F_2

" candidate itemsets for next scan

Use apriori-gen algo.

→ Apriori-gen algorithm (used to generate candidate item sets for next scan)

→ Some Phases: $F_{k-1} \times F_{k-1}$, but preserve lexicographic order

INSERT INTO C_k

SELECT $p.i_1, \dots, p.i_{k-1}, q.i_{k-1}$

FROM $F_{k-1} p, F_{k-1} q$

WHERE $p.i_1 = q.i_1, \dots, p.i_{k-2} = q.i_{k-2}$ and $p.i_{k-1} < q.i_{k-1}$

→ PRUNE phase: Based on monotonicity: Any subset of a frequent itemset must also be frequent.
FOR ALL itemsets $c \in C_k$ do
FOR ALL subsets with cardinality $k-1$ of c do
IF subset $\notin F_{k-1}$ do
DELETE c FROM C_k .

Have variable size itemsets in a hash-tree. Non-leaf nodes contain a hash table, with each entry in the bucket pointing to another node at the next level. Leaf nodes contain set of item sets.

→ No. of passes is proportional to the largest frequent itemset.

→ works for sparse matrices

Vertical Mining (VIPER)

→ Apriori uses row-wise & full data, whereas VIPER uses column-wise & bitmaps

→ Advantages of vertical organisation:

→ Natural for association rule mining's goal of discovering correlation

→ Easier to count support (bitwise AND)

→ No excess baggage from disk (why read rows which have a low support?)

→ Because calculations are synchronous, we can parallelise fully.

→ Advantages of bit representation:

→ Compression (LZ or Lempel encoding)

* Size of data will be larger than original data before compression

→ VIPER Algorithms

→ First pass

* R.L.E. gives worse results than huffman encoding

→ Snakes are created from the original horizontal database

→ Compression is done by swimming algorithm (based on huffman)

→ Compute F_1

→ Second pass

→ Computing F_2 is expensive, which is why we create a streaming database in memory; all pairs of items in each tuple are enumerated & sorted using a 2-D array.

→ F_2 is identified, but not written to disk

* Because candidate snake writing to disk is expensive & most of them do not satisfy support condition, we introduce lag of one pass in writing to disk. The snakes which are not written will be dynamically regenerated using snakes written to disk in previous pass.

Data lakes

→ Online Analytic Processing vs. Online Transaction Processing (OLAP vs. OLTP)

→ Relational OLAP (ROLAP)

→ The main reln. is called a fact table

E.g. Sales (pid, time, locid, sales)

Fact tables

are much

→ Each dimension may have additional attributes:

E.g. Products (pid, pname, cat, price)

larger than

dimensional tables.

→ OLAP Queries

→ Usually, we want to aggregate a measure over one or more dimensions.

→ Rollup = aggregate data

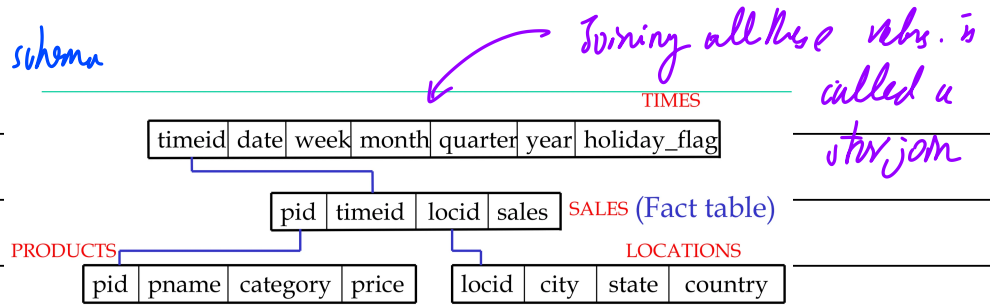
→ Drill-down = un-aggregate data (split)

→ Pivoting = aggregating on select dimensions

→ Slicing = Equality selection on ≥ 1 dimensions

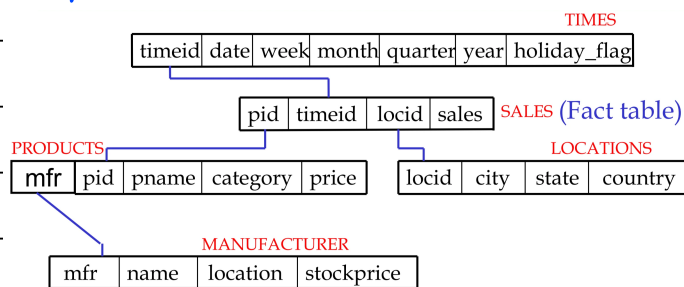
→ Dicing: range

→ Storage of cubes:
→ star schema



→ Only fact table is normalised; dimension tables need not be normalised due to their small size.

→ Snowflake schema



→ Even dimension tables are normalised (not to BCNF though)

→ Cube indexes

- Some are frequent b/t fact & dimension tables, which is why indexes are req'd.
- Space constraint will be there
- Because data is seldom updated, updating indexes is not an issue.

→ CUBE Materialisation: Pre-compute to save time.

→ Issues

- Which views to materialise? What index to use?
- Given a query and a few materialised views, can we rewrite the query in terms of the materialised views (so as to process query faster)?
- How often do the views need to be refreshed? Can it be done incrementally?

→ Given N tables, we have 2^N possible group-bys

→ Assumptions:

→ All queries are equally probable → No indexes

→ Query cost $\propto$ no. of rows examined

→ Algorithm:

$S = \{ \text{Top view} \}$ // Group by all N tables

for $i = 1 \rightarrow K$ do:

do a full traversal of $V - S$

all views (2^N)

no. of views that have not been materialised

K represents max.
views to be
constructed

select view $v \in S$ s.t. benefit(v, S) is max.

$S = S \cup \{v\}$

→ Benefit computation algorithm:

for all $w \leq v$ // all views that derivable from view v
including v itself

let $u = \text{view of least cost in } S \text{ s.t. } w \leq u$ // get the current
parent of w

if $C(v) < C(u)$ // if v provides some benefit to w

$$B_w = C(u) - C(v)$$

else

$$B_w = 0$$

total benefit of v including v = $\sum_{w \leq v} B_w$ // represents the total benefit to v
and all its descendants

→ Performance: * No better algorithm is possible!

$$\frac{B_{\text{greedy}}}{B_{\text{opt}}} \geq 1 - \left(\frac{n-1}{n} \right)^K \geq 0.63$$

→ Close to optimal if first view delivers most of the benefit

→ Equal " " " benefit of each successive view is the same.

binary heap space problem

→ Alternative: Total space is fixed, not no. of rows, hence benefit per unit space needs to be considered.

In-memory Databases

→ Cache vs. main mem. buffers:

→ Memory can be software controlled whereas cache is a black box

→ Cache misses may occur even if we have a near-empty cache, due to cache associativity.

→ Cannot trade CPU cycles for improving cache performance.

→ Techniques to get better locality (temporal/spatial)

→ Blocking (Use nested block join instead of nested tuple join)

→ Partitioning

→ Data filtering / clustering

→ Loop fusion

→ Data Structures: Vertical decomposition

→ Binary association table, one per column of original reln.
BAT

→ Partitioned joins:

→ Cluster both i/p relns. and create clusters that fit in memory

→ Join matching clusters

→ Radix join & partitioned hash join

"Item" Table

supplier	part	price	discount	qty	tax	flag	status	shipmode	date1	date2	comment
10	04.75	0.10					AIR				
10	11.50	0.00					MAIL				
11	10.20	0.00					TRUCK				
11	75.00	0.00					AIR				
11	02.50	0.00					SHIP				
12	92.80	0.10					AIR				
13	37.50	0.10					SHIP				
13	14.25	0.00					MAIL				

int int int float float int float char(1) int varchar date date date char(27)

width of relational tuple ≈ 80 bytes

vertical fragmentation in Monet

oid	int	oid	float	oid	float	oid	str	void	chr	encoding BAT
1000	10	1000	04.75	1000	0.10	1000	AIR		2	0 REG AIR
1001	10	1001	11.50	1001	0.00	1001	MAIL		3	1 TRUCK
1002	11	1002	10.20	1002	0.00	1002	TRUCK		1	2 AIR
1003	11	1003	75.00	1003	0.00	1003	AIR		2	3 MAIL
1004	11	1004	02.50	1004	0.00	1004	SHIP		6	4 RAIL
1005	12	1005	92.80	1005	0.10	1005	AIR		2	5 FOB
1006	13	1006	37.50	1006	0.10	1006	SHIP		6	6 SHIP
1007	13	1007	14.25	1007	0.00	1007	MAIL		3	

logical appearance optimized BAT storage: 1 byte per column

8 bytes 8 bytes 8 bytes

physical data structures

Cache conscious indexes

- Binary search and cache locality:
 - Temporal locality for initial accesses
 - Spatial " " " " " " " " (essentially doing binary search on a single cache line)
 - Poor hit ratio in the middle
 - Requires some computation (for calculating next index), and we absolutely cannot trade spatial localities.
 - Can we do better with some more memory?
- Cache sensitive search trees (CSS trees)
 - Eliminate pointers by embedding a complete search tree into an array.
 - Complete $(m+1)$ ary search tree
 - Entire tree needs to be rebuilt when an update occurs.
 - All nodes of the tree are stored contiguously, and nodes are accessed using offset from first node method.
 - Benefits over B^+ trees:
 - double no. of keys per node → fewer levels → faster search
- CSB (Cache sensitive B) trees:
 - Balanced multinary search tree
 - Tree of order d contains m keys, where $d \leq m \leq 2d$ keys
 - All child nodes of a node are in one node group. Node groups are stored contiguously and hence each node in a group can be accessed using offset method.
 - Instead of $(m+1)$ pointers, only one pointer is reqd.
(Eliminate most pointers, but not all. Unlike CSS trees, which eliminate all pointers)
 - Node structure
 - size of node = cache block size
 - Internal node
 - n_{keys} : no. of keys in the node ($d \leq n_{keys} \leq 2d$)

- first child: pointer to first child
- key list: array of size $2d$ to store list of keys
- Internal node
 - n_{keys} : no. of keys in the node ($d \leq n_{keys} \leq 2d$)
 - Set of $\langle \text{key}, \text{tuple} \rangle$ pairs
 - left/right sibling pointers.

→ Deletion is lazy (we will mark the deleted element as invalid)

→ Segmented CSB Trees:

→ Child nodes are divided into segments, and each segment is stored contiguously.

(Instead of all siblings being stored contiguously, we store only a few siblings)

→ No. of pointers reqd. = no. of segments

→ Benefit: when a node needs to split, copying data becomes easier.

→ Problem: Search becomes slower

→ Full CSB Trees:

→ Preallocate space for a full node group, reduces the need to call `malloc()` repeatedly.

→ Benefits:

→ Cheaper copying on split

→ Lesser `malloc()` calls, thereby lower overheads

→ However, leads to space overhead of 20%.

* $N = \text{no. of leaf nodes}$, $L = \text{cache line size}$, $m = \text{no. of slots in a node}$

→ Results:

→ Search

→ Update

		Cache Misses			Cache lines accessed on split
	B ⁺ Tree	$\frac{\log_2 N}{\log_2 m - 1}$		B ⁺ trees	2
	LSB Tree	$\frac{\log_2 N}{\log_2 (m+1)}$		(SB) trees	$(m-1) \cdot 2$
	(SB) Tree	$\frac{\log_2 N}{\log_2 (m-1)}$		(SB) trees (+ segments)	$\frac{(m-2+1) \cdot 2}{+}$
				(SB) trees (full)	$\frac{m-1}{2}$

→ Overall comparison

		Search	Update	Space	M.A. Overhead
	B ⁺ trees	slower	faster	medium	medium
	(SB) trees	faster	slower	lower	higher
	(SB) trees (+ segments)	medium	medium	lower	higher
	(SB) trees (full)	faster	faster	higher	lower

→ Conclusion

→ If we have enough space, choose full (SB) tree

→ " searches > updates, choose variants of (SB) trees

Columnar Databases (C-STORE)

→ Row stores are write optimised; best for OLTP.

→ Advantages of column store

→ Fetch only required columns. → Better cache efficiency

→ Due to data being homogenous, compression can be done better.

→ Disord antiques:

→ Requires addl. joins to reconstruct tuples

→ " more space to store tuple id's (reqd. to join tuples)

→ slower OLTP, due to multiple colors that need to be updated.

→ l-STORE

→ Columns are stored as a set of colors, which are pre-computed & stored.

→ Base tables are not stored physically.

→ Projections can be joined using the storage keys & join indexes.

→ To construct a base table from columns, multiple join indexes shall be reqd. Because join indexes are expensive to create & maintain, we make each col. part of several projections.

→ Compression of columnar data: Compression depends on datatype

→ Type 1: self-order, few distinct values

(value, position, no. of entries)

Unordered B tree index on value to position

E.g. $(4, 12, 7)$ indicates the values from posn. no. 12 to posn. no. $12 + 7 = 19$ is 4.

→ Type 2: Foreign order, few distinct values

(value, bitmap)

R.L.E.

Clustered B-tree on position to value.

→ Type 3: Self-order, many distinct values

Block Delta encoding: 1, 4, 7, 7, 8, 12
↓
1, 3, 3, 0, 1, 4

This is done for blocks (chunks) of data

only incremental values are stored

Clustered B-tree on block starting values

→ Type 4: Foreign-order, many distinct values

→ no encoding

Unclustered B-tree on values.

→ Redundancy: Recovery from log times too long, so no. too enough projections, to ensure K -safety, i.e. if K columns are lost, we still have all the data.

* worst case: recover from others in the network

→ Updates: Use write store

→ Column store, with the same projections & join indices as read store.

→ No compression

→ Each coln. is a collection of (values, storage key) tuples

→ Optimise read only queries → no locks

→ Each WS segment has an insertion & deletion vector

→ Concurrency control: 2PL

→ Recovery: Standard WAL w/ no-force & steal

→ Only log UNDO records

→ REDO is done using info from other sites.

→ Query optimisation

→ New operators:

→ Decompress: to get back the original data

→ Concat: Combine projections that are in same order

→ Permute: permute a projection based on join order.

→ Mask: Selects rows based on bitstring & projection

→ Cost-based query plan construction (System R style)

→ Used to choose the projection on which to run the query.

→ Compression cost is also accommodated for

Columnar indexes

→ Database cranking: Self-tuning database kernel

→ Cracker column: `SELECT A FROM R WHERE <some cond. on A>`

→ A selection operator on the column makes a copy of the column in memory. We perform a tuple-based physical re-organisation on this cracker column.

→ Each cracker column has an AVL tree to maintain partitioning information.

→ Cracker map `SELECT B FROM R WHERE <some cond. on A>`

→ Values of A are stored in left index, values of B are stored in right index.

→ Cracking is done on the whole projection

→ All cracking done on M_{AB} must be reflected on M_{A*} . We can delay these cracks until a query is called w.r.t. M_{A*} .

→ We keep records of these cracks in a cracker type.

Extra: Algo for Contended read/writes select operator:

- 1 If there is no cracker type on A, create it
- 2 " " " " " map: AB, " "
- 3 Align M_{AB} using T_A // make M_{AB} catch-up with other M_{A*}
- 4 Search M_{AB} to find the area w of the pieces related to the condn. on A
- 5 If we cannot find the answer using existing boundaries,
 - a Physically re-organise w to move false bits out of the contiguous area
 - b Update the cracker index
 - c Append the predicate to the cracker type.
- 6 Return a non-materialised view of the tail of w

might when.

The locations of the attributes (i.e. pointers) are sent instead of the actual values.

→ Multiple select queries $\text{SELECT D FROM R WHERE } \langle \text{some condn. on A, B, \&C} \rangle$

→ Choose a selection predicate, e.g. A

→ Choose the selection predicate that the condition on it is the most selective.

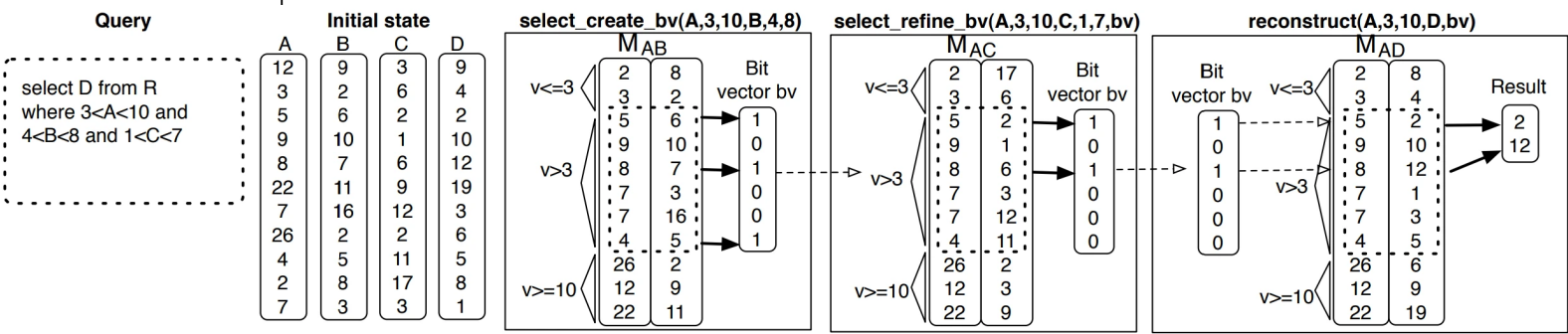
→ This can be found using cracker indices.

→ Choose the most aligned map M_{A*} (map with most cracks, i.e. some one who is at the end of the cracker type) // for accurate statistics

→ Estimate cardinality interval of the predicate using index boundaries.

→ Create cracker maps M_{AB} , M_{AC} & M_{AD} (if not already made)

→ Create a bit vector for B that identifies the tuples satisfying the condn. on B. Repeat for C.
 → AND the two bit vectors with Map to get the result



→ Many other operators, i.e. MIN/MAX can exploit the vectors by only searching the relevant vectors.

→ Partial maps

→ Do not materialise an entire map; choose to materialise in chunks

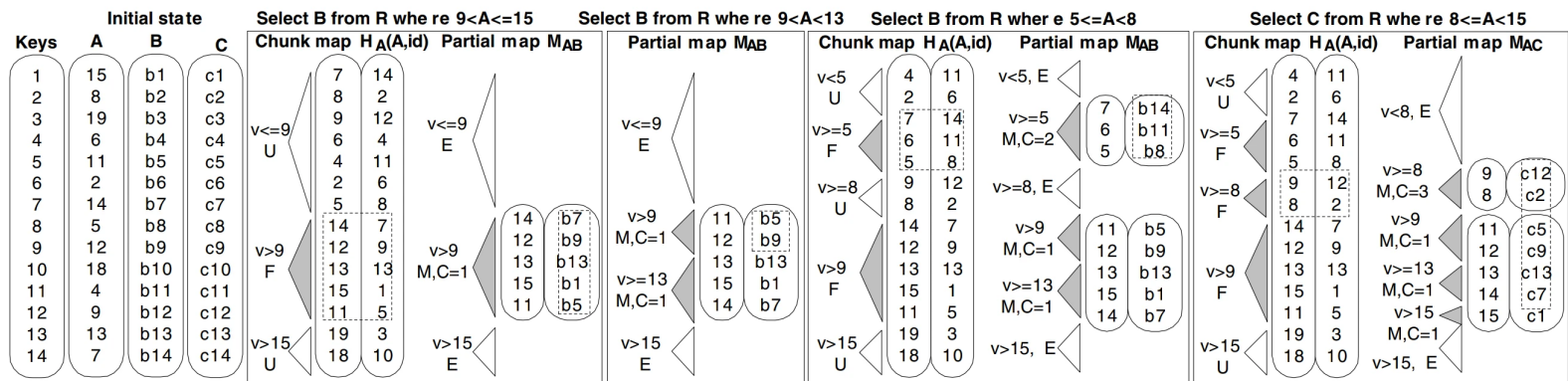


Figure 8: Using partial maps (U=Unfetched, F=Fetch, E=Empty, M=Materialized, C=ChunkID)